

HW3

Program Information:

I have uploaded my C++ project to the GitHub for this class (https://github.com/tgmaxson/HPC_TGM) under folder "hw3". This is a modified form of HW1, which now supports OpenMP.

Cluster Information:

I am running on Ubuntu 22.04.2 LTS via a cluster known as "Clustie" to our research group, which I have full access to. Tests are run on a test node which has nothing else running at the time of execution, except for SLURM and other background tasks which are default to Ubuntu Server installs. GCC is version 11.4.0 and ICC is version 2021.10.0 via OneAPI released on 06/09/2023. CUDA is provided via Nvidia HPC SDK version and nvcc is version *cuda_12.0.r12.0/compiler.32267302_0* with CUDA version V12.0.140.

System Hardware:

Motherboard: H11SSL-i

CPU: Single Socket 32 Core AMD Epyc 7551P, 2.0 GHz, boost to 3.0 GHz. SMT Disabled.

Memory: 256 GB of ECC DDR4-2400 via 8 x 32 GB sticks in 8 channel

GPU: 2 x MSI Suprim Liquid X24G RTX 4090 with 24GB of memory.

Profiling:

Code performance is profiled using `bash time`, `valgrind`, or `clock_gettime` from `<time.h>`, surrounding the majority of the code to get real time clock performance. `clock_gettime` allows for this to be output with the program results and agrees with all other methods since the only missing call before finishing is the final print statement. Threading is controlled via the `omp_set_num_threads()` command. Disabling OpenMP entirely makes the program run much faster as seen in the table below.

OpenMP:

Performance analysis of OpenMP appears incorrect. In all cases, adding threading is slower than without it. This is true across multiple clusters and indicates for some reason my program has issues with threading. An analysis of speedup follows, but it is not a speedup as a result.

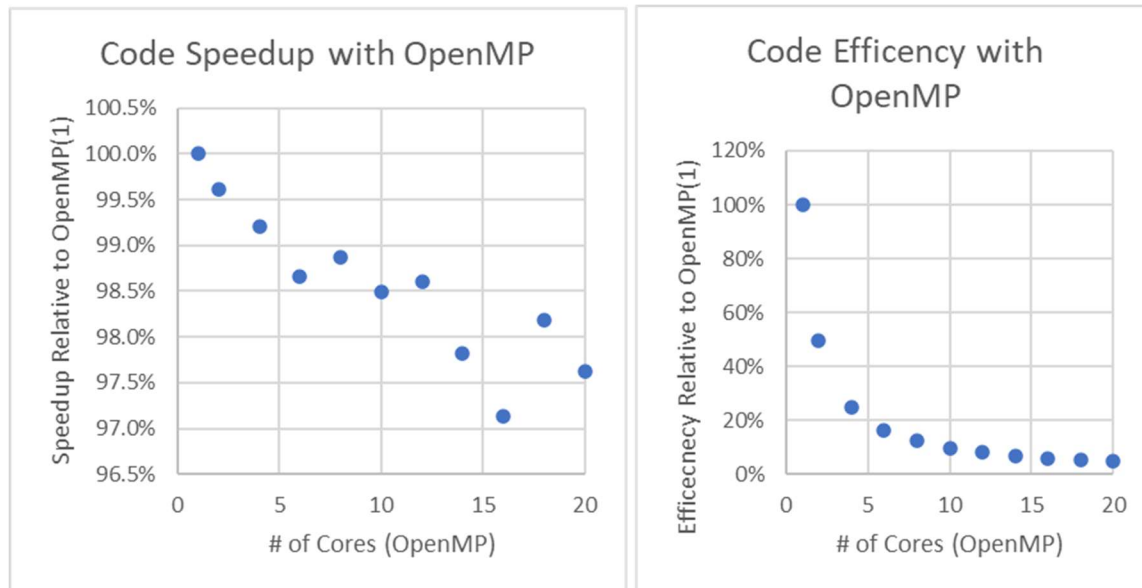


Figure 1: Pictured on the left is the ‘speedup’ with OpenMP relative to a single thread, with the efficiency per core pictured on the right.

Table 1: Raw data in Figure 1 and the non-threaded version.

Parallelization	Threading	Time	Speedup	Efficiency
N/A	1	1060.898	234.4%	234%
OpenMP	1	2486.915	100.0%	100%
OpenMP	2	2496.715	99.6%	50%
OpenMP	4	2506.961	99.2%	25%
OpenMP	6	2520.728	98.7%	16%
OpenMP	8	2515.197	98.9%	12%
OpenMP	10	2524.813	98.5%	10%
OpenMP	12	2522.229	98.6%	8%
OpenMP	14	2542.405	97.8%	7%
OpenMP	16	2560.445	97.1%	6%
OpenMP	18	2532.996	98.2%	5%
OpenMP	20	2547.392	97.6%	5%

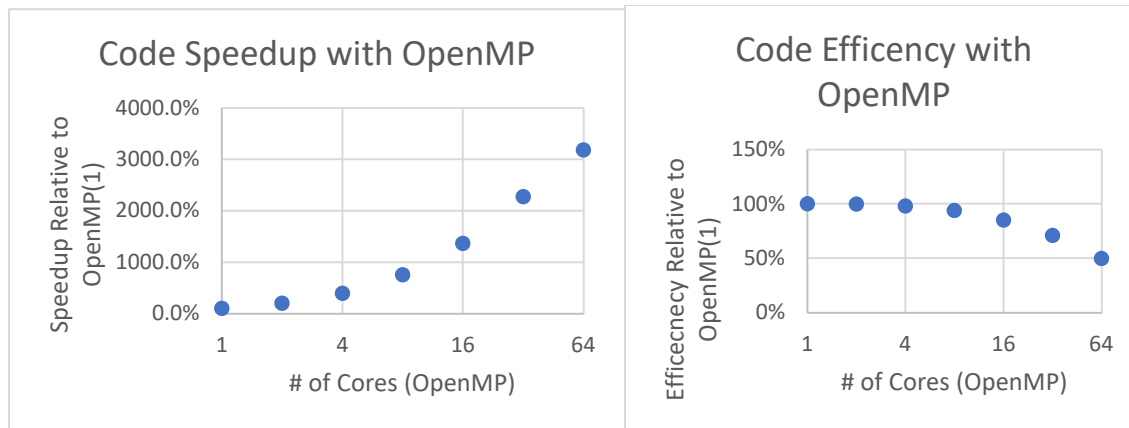


Figure 2: Pictured on the left is the ‘speedup’ with OpenMP relative to a single thread, with the efficiency per core pictured on the right for the matrix program.

Table 2: Raw data in Figure 1 and the non-threaded version for matrix program.

Parallelization	Threading	Time	Speedup	Efficiency
N/A	1	0.932	102.4%	102%
OpenMP	1	0.954	100.0%	100%
OpenMP	2	0.478	199.6%	100%
OpenMP	4	0.243	392.6%	98%
OpenMP	8	0.127	751.2%	94%
OpenMP	16	0.07	1362.9%	85%
OpenMP	32	0.042	2271.4%	71%
OpenMP	64	0.03	3180.0%	50%

Results:

It is clear from the table that simply enabling the OpenMP flags causes a large issue in the performance of the program, but a slight slowdown when enabling it shouldn’t be completely unexpected. However, it seems something is wrong as the speedup when increasing the number of threads consistently decreases until 16 cores, where the performance seems inconsistent beyond this point. The efficiency of the program is directly $1/\text{cores}$ as the speedup is never favorable.

Diffing the output files together confirms that the outputs are identical across all tests with and without OpenMP. Small tests confirm that the early break condition works, which was confirmed with the presence of a print statement not present in the final version.

These results indicate that something needs to be redone in this code to make it work correctly.

I also compiled a supplementary portion of the HW as HW3-matrix which shows the following performance using OpenMP and SIMD to multiply matrices together. The size is hard coded at 512x512 for this test and relatively small, but even shows a speedup as seen in Figure 2. This largely indicates to me that something must be wrong in my program specifically.

Compiler Flags:

Tristan Maxson

None of the compiler flags seem to allow for the code to run faster using multiple threads. It is unclear to me why this is. Using O3 (most of the document) gives the best performance and O2 is almost equivalent (95% of speed of O3). O1 and O2 are approximately 55% of the speed of O3. Enabling debug flags also degrades performance to approximately O0 or 45% of the speed of O3. All these tests are performed on a 5000x5000 grid for 5000 iterations.

For the matrix program, O3 is used on its own.